TTS Evaluation

Dan DeGenaro
Georgetown University
drd92@georgetown.edu

Abstract

In this assignment, we evaluate a neural audio codec as well as six text-to-speech systems (five open-source, one API-based) and introduce a TTS component into the chatbot pipeline.

1 Introduction

1.1 Importance of TTS quality and audio codecs in conversational AI

Conversational AI systems (those capable of both interpreting and producing spoken language) need to be able to reliably generate naturalistic speech sounds. Users are likely to abandon use of a system whose synthesized speech is incomprehensible, riddled with grammar or speech errors, or rife with distracting or grating artifacts. As such, the development of robust TTS systems, capable of handling specialized vocabulary and producing naturalistic-sounding speech while still giving a comfortably low latency, is of great importance.

Techniques employing neural audio codecs (systems that develop low-dimensional, quantized, machine-learned representations of audio data to compress and reconstruct it efficiently) (Zeghidour et al., 2022; Défossez et al., 2023; Kumar et al., 2023) have made great strides by sharply reducing the dimensionality of audio data before processing it, as well as vector quantization techniques like residual vector quantization (RVQ) in order to discretize the signal, making it easier to model with language-modeling like techniques (Rosenfeld, 1996) over discrete audio token vocabularies.

1.2 Research questions

How do neural codecs balance compression and quality? We examine one popular neural audio codec, FAIR's Encodec model (Défossez et al., 2023), finding that the differences between audio reconstructed using 8 versus 16 codebooks are hard to perceive, while using fewer than 8 codebooks introduces noticeable artifacts, which are least pronounced for sound effects and most pronounced for music, with speech falling in the middle.

Which TTS systems provide the best quality-latency trade-off? We examine five open-source TTS systems, namely Bark (Suno AI, 2025), MMS (Pratap et al., 2024), Parler-TTS (Lyth and King, 2024), Piper (OHF-Voice, 2025), and VALL-E X (Zhang et al., 2023). We also evaluate Google Translate's API-based TTS system using the gTTS library (Durette, 2025). We find that while Bark provides most naturalistic-sound speech, Piper is a much better alternative due to being an order of magnitude faster on a Mac MPS device, and due to Bark generally having annoying artifacts. Piper also achieves similar latencies to the API-based gTTS system, while sounding more naturalistic (though occasionally making speech errors, and sounding just slightly robotic). At the lower end of the rating scale, Parler-TTS tended to have significant speech errors and intonation problems, while VALL-E X had significant artifacts, speech errors, and inconsistent voices (some of which were grating). MMS tended to sound obviously robotic, and was completely unable to handle simple cases such as numbers.

1.3 Overview of neural codec architectures

SoundStream (Zeghidour et al., 2022) developed a convolution-based encoder-decoder architecture, with RVQ employed at the end of the encoder. The convolutional encoder learns a downsampled representation of the input audio features, and the RVQ module effectively clusters and merges the downsampled audio feature vectors, yielding a sequence of discrete audio "token embeddings," where similar audio samples will be represented

by identical vector representations. Repeating this process several times, in a hierarchical fashion, involves training a sequence of "codebooks," which are employed in order to hierarchically cluster and merge audio feature embeddings into smaller and smaller token spaces until a desired bitrate is achieved. Using more codebooks yields less compression, but better reconstruction performance.

SoundStream was trained to minimize the distance between the original and reconstructed waveforms and their multi-scale short-time Fourier transforms (STFTs). Additionally, a discriminator was trained alongside SoundStream in an adversarial setup to encourage SoundStream to produce more perceptually faithful reconstructions of audio.

Encodec (Défossez et al., 2023) improves upon SoundStream by adding an RVQ commitment loss, penalizing RVQ choices that cause the quantized representations to stray significantly from the unquantized ones. It additionally introduces a more intricate architecture for the discriminator for more robust adversarial training. They additionally experiment with a lightweight transformer (Vaswani et al., 2017) with a language modeling objective over the discrete audio tokens. The authors also evaluate Encodec on a broader variety of audio than the authors of SoundStream, and achieve better compression and reconstruction quality. The Descript Audio Codec (DAC) is a popular open-source alternative constructed similarly (Kumar et al., 2023).

1.4 Evolution of TTS systems

Speech synthesis systems can be traced back to Von Kempelen's mechanical speaking machine, which he introduced in 1769. The Bell Labs Voder was the first electrified speech synthesis system, which required a human to play it, somewhat like a musical instrument. Both systems relied on what is now termed the "source-filter" theory of speech production, namely that speech sounds result from various methods of obstructing an airflow in the vocal tract.

The first automatic speech synthesis system (Klatt, 1980) synthesized speech using a phoneme-based lookup and compositions of formants for vowels. Later, (concatenative synthesis) and linear prediction systems came to dominance in the 1990s (Kubichek, 1993), wherein locally-extracted cepstral coefficients were used to attempt to predict speech segments using simple, statistical linear models. In the 2000s, more advanced machine learning techniques such as HMMs (Zen et al., 2009) gained in popularity, before the deep learning revolution finally made itself known in speech synthesis with WaveNet (van den Oord et al., 2016) and the Tacotron models (Wang et al., 2017; Shen et al., 2018).

2 Methods

2.1 Hardware specifications

All experiments with local models are conducted on a 16 GB Apple M4 chip running MacOS Sequoia 15.7.2, whose memory is shared by the CPU and MPS device, with all computation handled by the MPS system when an interface is exposed to achieve this. This is notably not possible for Piper, which abstracts this away, and for gTTS (an API-based system, where computation is not done locally).

2.2 Neural codec evaluation

We select the 48 kHz-input variant of Encodec (Défossez et al., 2023) to evaluate due to its improvements over SoundStream (Zeghidour et al., 2022) and its open-source implementation being readily available on HuggingFace.

We collect a diverse collection of ten royalty-free audio samples from Pixabay (Pixabay, 2025)[1] for the purpose of evaluating Encodec's abilities to compress and reconstruct various audio files, which we term:

- `birds.mp3` - a cacophony of bird calls.
- `glass.mp3` - the sound of shattering glass.
- `leaves.mp3` - dry leaves being crunched as they are walked on.
- `muffled.mp3` - muffled, incomprehensible speech.
- `one.mp3` - a slightly distorted man's voice uttering the word "one."
- `rap.mp3` - a Bengali rap song.
- `robot.mp3` - a compressed, robotic voice uttering the words "does not compute."

[1] We cannot redistribute this audio per license agreements. Similar audio can be easily found on Pixabay.

- `thank_you.mp3` - a person offering thanks in several ways.
- `trap.mp3` - an instrumental trap beat.
- `wow.mp3` - a woman's voice uttering the word "wow."

We evaluate the chosen Encodec implementation using 2, 4, 8, and 16 codebooks (these are all the available options in the encoder's inference pipeline, corresponding to target bandwidths of 3.0, 6.0, 12.0, and 24.0 Kb/s, respectively).

For each codebook setting, we compress each of the ten audio files into an encoder representation, and write it to a JSON file to see what near-optimal compression of the audio file is achieved by the encoder (including the RVQ process). We then pass the encoded representation through the decoder, and reconstruct the original audio. We compute standard metrics of compression for every pairing of codebook count and audio file:

- Audio compression ratio (reconstructed audio file size divided by original audio file size).
- Encoder compression ratio (JSON representation of encoding's file size divided by original audio file size.
- Reconstructed audio bitrate (Reconstructed audio file's size in bits divided by its length in seconds).

2.3 TTS system evaluation

We examine five open-source TTS systems, namely Bark (Suno AI, 2025), MMS (Pratap et al., 2024), Parler-TTS (Lyth and King, 2024), Piper (OHF-Voice, 2025), and VALL-E X (Zhang et al., 2023). We also evaluate Google Translate's API-based TTS system using the gTTS library (Durette, 2025). These were selected due to ease of installation and confirmation during initial testing that they each gave somewhat reasonable, audible outputs.

Bark Bark (Suno AI, 2025) uses several scaled-down, transformer-style architectures similar to GPT (Radford et al., 2018) - one for "semantic" features, one for "fine-grained" features, and one for "coarse-grained" features, along with an Encodec model which comprises a large portion of Bark's parameters.

MMS MMS (Pratap et al., 2024) uses primarily convolutional layers in a highly modularized architecture, with a transformer-based text encoder. It uses a HiFi-GAN-like (Kong et al., 2020) architecture as its decoder.

Parler-TTS Parler-TTS Mini v1 (Lyth and King, 2024) uses a pretrained T5-based (Raffel et al., 2020) text encoder and uses the DAC as its audio encoder (primarily convolution-based), and finally a custom autoregressive transformer decoder for generation.

Piper Piper (OHF-Voice, 2025) is implemented in ONNX and it is therefore a bit of a pain to analyze its architecture in much detail. Piper's code uses some file path operations that are not supported by older versions of Python. Pipe sometimes raises a circular import error, but can be run via an interactive session.

VALL-E X VALL-E X (Zhang et al., 2023) uses Encodec in conjunction with an autoregressive language model to generate discrete audio tokens from joint phoneme- and audio-based representations, which are decoded by Encodec to produce speech. VALL-E X uses an outdated call to load weights of its PyTorch models, which needed to be fixed.

gTTS gTTS (Durette, 2025) is an API for Google Translate's TTS system of unknown architecture. gTTS sometimes raises a circular import error, but can be run via an interactive session.

2.4 Evaluation strategy

We use the following 15 prompts, which aim to test a diverse mix of common failure modes of TTS systems.

- `The weather today is sunny and warm.` (basic)
- `Although the meeting was scheduled for Tuesday, the unexpected snowstorm forced us to reschedule for the following week.` (syntactically complex)
- `She sells seashells by the seashore.` (tongue twister)
- `How much wood would a woodchuck chuck?` (tongue twister)
- `On March 15th, 2024, the temperature reached 72.5 degrees.` (numbers)

- `2050 is 25 years from now, and a lot more than 2,050 seconds from now.` (numbers, comma matters)
- `Where are you going?` (question)
- `You're going where?` (question via intonation)
- `I can't believe you did that!` (negative exclamation)
- `Congratulations on your achievement!` (positive exclamation)
- `The algorithm uses stochastic gradient descent for optimization.` (technical terms)
- `The bandage was wound around the wound.` (context-based pronunciation)
- `I need to present the present.` (context-based pronunciation)
- `Where should I lead the lead balloon?` (context-based pronunciation)
- `Supercalifragilisticexpialidocious.` (long nonce word)

The annotation system presents prompt-matched pairs (both items in the pair are from the same prompt) of synthesized speech audio. The order of the prompts is randomly shuffled, and every pair of models is compared head-to-head, with all prompt-matched pairs for a given pairing of models shown consecutively. The annotator may replay both audio files as many times as needed before committing to the better choice. The annotator judged audio files primarily on naturalness and freeness of mistakes, with artifacts being considered acceptable if not overly distracting.

Latencies are always measured with the text prompt pre-loaded, before the measurement begins. The latency measurement includes any preprocessing done on the text, processing of the encoded text by the TTS model, and the writing of the model's output audio to a file.

3 Results

3.1 Codec evaluation results

We show the compression ratios, by audio file size, achieved by Encodec, in Figure 1. These ratios

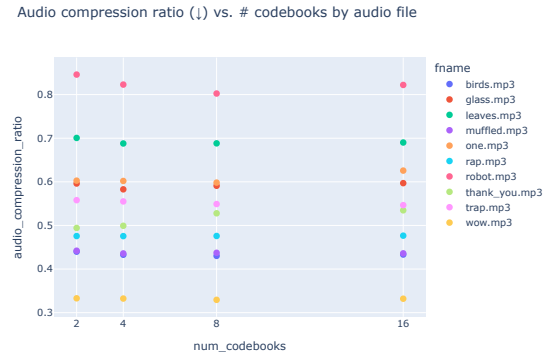

Figure 1: Compression ratios achieved by the entire model (reconstructed audio's file size as a portion of the original audio's).

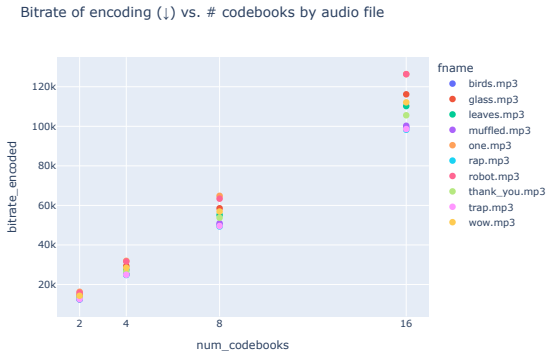

Figure 2: Compression ratios achieved by the encoder (JSON encoding file size as a portion of the original audio's).

are computed by dividing the file size of the reconstructed audio file by the size of the original audio file, with better compression therefore being closer to 0, and decompression being indicated by a ratio greater than 1.

Similarly, we show the compression ratios achieved by Encodec's encoder in Figure 2. These are computed similarly, but with the numerator now being a JSON representation of the encoder's outputs, which are typically much smaller than the reconstructed audio output by the decoder.

We analogously show bitrates for both kinds of compressed files, where the bitrate is computed as the size of the compressed file in bits, divided by the reconstructed audio's duration in seconds. Figure 3 shows the bitrate of the reconstructed audio files, while Figure 4 shows the bitrate of the JSON encodings, which is somewhat nonsensical (the JSON file has no "duration," so we again rely

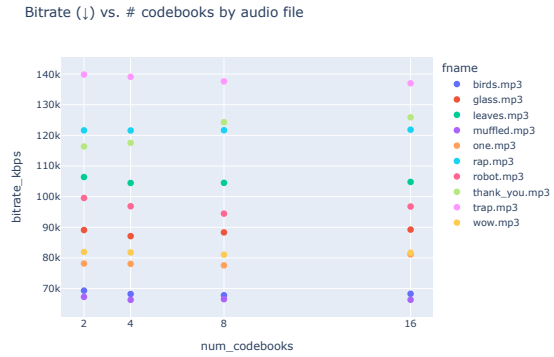

Figure 3: Bitrates achieved by the entire model (reconstructed audio's file size in bits divided by its duration).

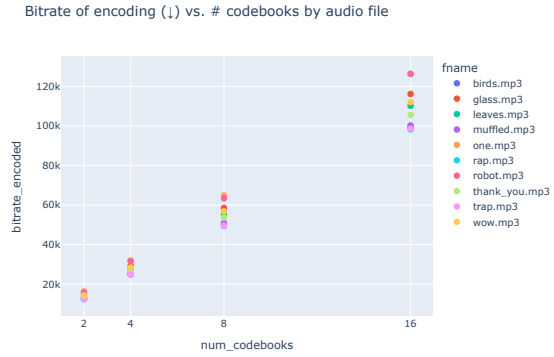

Figure 4: Bitrates achieved by the encoder (JSON encoding file size in bits divided by the reconstructed audio's duration).

on the reconstructed audio's duration).

Notably, the compression savings with fewer codebooks are really achieved only by the encoder representations, but are lost when decoding and reconstructing the audio again. That said, the reconstructed audio is consistently smaller than the original audio in all cases.

An interesting finding is a fairly consistent ranking of the compression rates achieved *for each file.* It would seem that files that are more compressed to begin with suffer from higher (worse) compression ratios, which makes sense. A file that is already rather compressed is hard to compress much further. Following this line of thinking, it would appear that `robot.mp3`, `leaves.mp3`, and `one.mp3` are more compressed to begin with, while files such as `wow.mp3`, `birds.mp3`, and `muffled.mp3` are very uncompressed and ripe to be compressed. The speech and music tend to be less compressed to begin with than the sound effects, due to the need for high sampling rates and precision to capture the broad frequency spectra produced by those sounds in comparison to sound effects.

Qualitatively, this manifests in the audible degradation of quality and introduction of artifacts at lower numbers of codebooks. Across all audio files, there is no appreciable/audible difference between the reconstructed audio which was compressed with 8 codebooks versus 16, but notable artifacts are present with 4, and even more severely so at 2. Therefore, 8 codebooks is a sufficient performance-compute tradeoff.

There are a few exceptions, however. For `robot.mp3` in particular, the fidelity is maintained quite well even with just 4 codebooks due to already-compressed nature of the original audio file. Other sound effects such as `leaves.mp3` are also fairly artifact-free at 4 codebooks, but not necessarily as faithful to the original.

Model	Wins	Losses	Win% (↑)	Elo (↑)
Bark	**62**	**13**	**83**	**1793**
Piper	43	32	57	1574
gTTS	42	33	56	1538
Parler	39	36	52	1483
Vall-E X	33	42	44	1404
MMS	6	69	8	1208

Table 1: Models by win rate and elo.

3.2 TTS Results

Table 1 shows each model by win rate and elo, which gives a consistent ranking of models from being chosen the most during A/B testing to least, across all pairs involving generations from a given model. We can also see the mean latency over the 15 prompts broken down by model in Figure 5.

4 Discussion

4.1 TTS System Ranking and Selection

Given that Piper is faster than even an API-based system, and second in performance only to Bark (which has more notable artifacts at times, and is also the slowest model by far), we select Piper as the model with the best performance-latency tradeoff, and therefore adopt this model for our chatbot pipeline. gTTS is a suitable API-based backup. All other systems are functionally not usable due to long latency (Bark, Parler, and VALL-E X are unacceptably slow), noticeable artifacts (Bark and VALL-E X), inconsistent voices (Bark and VALL-

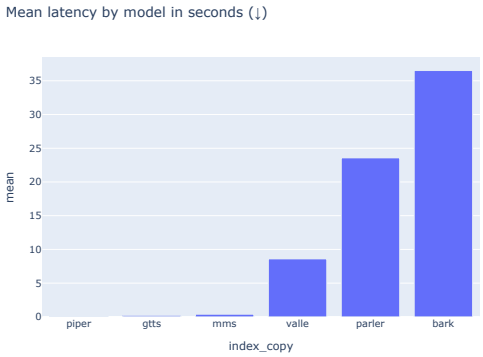

Figure 5: Mean latency averaged over the same 15 prompts by model.

E X), or significant speech errors (MMS, VALL-E X, Parler). In terms of deployment, only Piper and gTTS are worth considering, with Piper being a better choice due to being ranked slightly above gTTS in terms of performance, being slightly faster, and being open-source (i.e. no rate limiting or potential cost incurred, though gTTS is free).

4.2 Neural Codec Insights

In general, a 12 Kb/s target bandwidth (achieved using 8 codebooks) seems to be the optimal performance-compute tradeoff, as there is little appreciable difference between 12 Kb/s and 24 Kb/s (8 and 16 codebooks, respectively), across sound effects, speech, and music. Noticeable artifacts tend to develop even for sound effects at 6 Kb/s (4 codebooks), so the optimal tradeoff is the same for all audio. In a practical deployment setting such as a speech-enabled chatbot, 8 codebooks is probably plenty. However, if dealing with audio that should be extremely high-fidelity, such as professional recordings for audiobooks, movies, music, TV, or film, the maximum number of codebooks (16) should be employed to preserve the highest audio quality possible, in case artifacts are possibly occasionally introduced with fewer codebooks. TTS systems benefit from RVQ systems that use more codebooks as they can more accurately capture variability in audio data, leading to smoother, artifact free speech synthesis.

4.3 Practical Deployment

Conversational AI systems need TTS systems with very low latency, but with appreciable ability to correctly synthesize speech for technical terms, numbers, tongue twisters, ambiguously pronounced words, etc. A system like Piper is quite practical for an enthusiast deployment, as it is fast, free, runnable locally, and uses minimal memory.

5 Integration with Chatbot

We integrate the Piper TTS system into the chatbot system successfully. We find little difference between latencies with or without TTS employed, as Piper is incredibly fast. Introducing it has no appreciable impact on latency as compared to the LLM generation overhead, which dominates the running time of a response.

6 Conclusion

In summary, neural codecs are a great tool for compressing audio and rather faithfully reconstructing it later on. For the Encodec architecture, 8 codebooks (a targeted 12 Kb/s bandwidth) appears to be optimal for most use cases, except when extremely high-fidelity audio is desired, such as when compressing pre-recorded, professional multimedia, when it might make sense to use 16 (24 Kb/s) for certainty.

Many open-source TTS systems suffer from poorly documented and maintained research code-style implementations. Many open-source models are quite slow, introduce undesirable artifacts, and produce inconsistent voices. Piper TTS is a standout system for its usability, unbelievably low latency, and performance on rare words, numbers, etc. It sounds more naturalistic (though still robotic) than Google Translate's TTS API system, and has an even lower latency when locally deployed. Additionally, it produces a consistent voice, with the voice embedding being modularized to yield different voices via the same TTS system architecture. It is an excellent workhorse for TTS, and is a logical choice for any deployment scenario where open-sourcing, data privacy (all local systems) and/or cost effectiveness come into play.

6.1 Limitations

This study is limited in many ways. We evaluate only one neural codec on a very small number of audio samples, with no quantitative metrics employed. We additionally evaluate six TTS systems, but fairly qualitatively, with a single annotator, over a limited sampling of just 15 generations per system. We learned that TTS evaluation is primarily based on aggregation of human judgments

over many samples, but this often does not account for things like latency, which is just as important in real-world deployment scenarios.

References

Pierre Nicolas Durette. 2025. pndurette/gTTS. Original-date: 2014-05-15T02:52:58Z.

Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. 2023. High Fidelity Neural Audio Compression. *Transactions on Machine Learning Research*.

Dennis H. Klatt. 1980. Software for a cascade/parallel formant synthesizer. *The Journal of the Acoustical Society of America*, 67(3):971–995.

Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. 2020. HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis. In *Advances in Neural Information Processing Systems*, volume 33, pages 17022–17033. Curran Associates, Inc.

R. Kubichek. 1993. Mel-cepstral distance measure for objective speech quality assessment. In *Proceedings of IEEE Pacific Rim Conference on Communications Computers and Signal Processing*, volume 1, pages 125–128 vol.1.

Rithesh Kumar, Prem Seetharaman, Alejandro Luebs, Ishaan Kumar, and Kundan Kumar. 2023. High-Fidelity Audio Compression with Improved RVQGAN. *Advances in Neural Information Processing Systems*, 36:27980–27993.

Dan Lyth and Simon King. 2024. Natural language guidance of high-fidelity text-to-speech with synthetic annotations. *arXiv preprint*. ArXiv:2402.01912 [cs].

OHF-Voice. 2025. OHF-Voice/piper1-gpl. Original-date: 2025-03-28T21:47:10Z.

Pixabay. 2025. 5.8 million+ Stunning Free Images to Use Anywhere - Pixabay.

Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Sayani Kundu, Ali Elkahky, Zhaoheng Ni, Apoorv Vyas, Maryam Fazel-Zarandi, Alexei Baevski, Yossi Adi, Xiaohui Zhang, Wei-Ning Hsu, Alexis Conneau, and Michael Auli. 2024. Scaling Speech Technology to 1,000+ Languages. *Journal of Machine Learning Research*, 25(97):1–52.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Ronald Rosenfeld. 1996. A maximum entropy approach to adaptive statistical language modelling. *Computer Speech & Language*, 10(3):187–228.

Jonathan Shen, Ruoming Pang, Ron J. Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, Yu Zhang, Yuxuan Wang, Rj Skerrv-Ryan, Rif A. Saurous, Yannis Agiomvrgiannakis, and Yonghui Wu. 2018. Natural TTS Synthesis by Conditioning Wavenet on MEL Spectrogram Predictions. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4779–4783. ISSN: 2379-190X.

Suno AI. 2025. suno-ai/bark. Original-date: 2023-04-07T19:34:48Z.

Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. 2016. WaveNet: A Generative Model for Raw Audio. In *Proc. SSW 2016*, pages 125–125.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Yuxuan Wang, R. J. Skerry-Ryan, Daisy Stanton, Yonghui Wu, Ron J. Weiss, Navdeep Jaitly, Zongheng Yang, Ying Xiao, Zhifeng Chen, Samy Bengio, Quoc Le, Yannis Agiomyrgiannakis, Rob Clark, and Rif A. Saurous. 2017. Tacotron: Towards End-to-End Speech Synthesis. In *Proc. Interspeech 2017*, pages 4006–4010.

Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2022. SoundStream: An End-to-End Neural Audio Codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507.

Heiga Zen, Keiichi Tokuda, and Alan W. Black. 2009. Statistical parametric speech synthesis. *Speech Communication*, 51(11):1039–1064.

Ziqiang Zhang, Long Zhou, Chengyi Wang, Sanyuan Chen, Yu Wu, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, Lei He, Sheng Zhao, and Furu Wei. 2023. Speak Foreign Languages with Your Own Voice: Cross-Lingual Neural Codec Language Modeling. *arXiv preprint*. ArXiv:2303.03926 [cs].